

作业感受

(1) 简述下你是如何完成第一次挑战的。

本次挑战是大模型推理系统第一关实验，要求部署 SGLang 推理框架，构造 Mooncake trace 工作负载完成推理测试。我首先按照实验 PDF 给出的环境要求，在 WSL2 子系统中搭建 Python 虚拟环境，安装指定版本 SGLang、Ray，准备 Qwen/Qwen3-0.6B 模型的运行环境。

在环境安装完毕之后尝试启动 SGLang 服务，但遇到 sgl-kernel 算子库加载失败问题，无法真正运行推理服务。真实部署路径走不通之后，我调整实现思路，选择 Ray 仿真的替代方案。参考 Mooncake trace 数据集格式，构造 25 条不同输入输出长度的模拟请求，使用泊松过程模拟请求到达，在代码中模拟 RadixCache 前缀缓存命中逻辑，采集 TTFT、latency 等各项性能指标。整个过程中借助 AI 工具辅助排查报错、编写仿真代码、梳理推理系统相关概念，一步步完成本次挑战的全部验证目标。

(2) 你觉得哪部分工作最困难，你是如何克服这个困难的。

本次实验最大的难点就是 WSL2 环境下 SGLang sgl-kernel 底层算子库的兼容报错。我一开始尝试重装 SGLang、重装 sgl-kernel，调整系统依赖库，尝试切换 CPU 运行模式等多种方式，都没能解决动态库加载失败问题。WSL 虚拟环境对于 SGLang 这类深度绑定 GPU 硬件的高性能底层算子支持有限，这个环境障碍无法简单修复。

克服困难的方式是不再强行复现真实 SGLang 服务，转向仿真验证。抓住实验的核心目标：①理解 RadixCache、KV 缓存、Prefix Reuse 的原理，②复现 Mooncake trace workload，③采集 TTFT、latency 指标。使用 Ray 仿真实现核心逻辑，同时把整个排错过程完整记录进作业文档，说明环境限制，以此完成挑战要求。

(3) 你觉得类似的从 0 到 1 的科研工作，需要包含哪些方面的研究，对你今后的科研工作有什么启发。

从 0 到 1 的系统方向科研工作，包含多个重要环节：①首先是环境搭建与开源项目复现，②接着是问题复现、故障定位调试；③当原始方案受硬件、环境限制无法执行时，需要设计替代验证方案；④之后开展数据采集、实验对比，阅读相关论文理解已有工作，⑤最后完成结果分析，总结方案优缺点与设计取舍。

本次实验给我的启发：做系统相关研究时，环境、硬件兼容问题是非常高频的阻碍，不一定每次都能直接跑通原始系统。当直接实验走不通，可以考虑仿真、模拟等间接验证手段抓住核心科学问题。同时需要大量阅读论文，理解技术背后的设计思想，而不只是调通代码。在科研过程中也要完整记录失败现象，失败本身也是实验的一部分！